Implementación comparativa de la estructura de una base de datos

Red Social Estudiantil Pascualina

Integrantes:
Mariana Díaz Lastre
José David Celis Cepeda

Grupo:
7

Asignatura:
Bases de Datos I (G100)

Docente:
Oralia Cortes Grajales

Institución Universitaria Pascual Bravo
2026

Introducción

Este informe documenta la adaptación física de la base de datos de la Red Social Estudiantil Pascualina en PostgreSQL, MySQL y Microsoft SQL Server. Se toma como punto de partida el modelo de la Tarea 3, que define diez tablas para estudiantes, perfiles, publicaciones, grupos, eventos, intereses y sus relaciones. La implementación conserva las entidades y reglas principales, y adapta la sintaxis y los tipos de datos a cada motor.

Selección de los SGBD y justificación

PostgreSQL

Se conserva PostgreSQL porque el equipo ya cuenta con una propuesta inicial y porque es un motor de código abierto con restricciones CHECK, claves foráneas, transacciones y soporte avanzado para datos semiestructurados mediante JSONB. El índice GIN permite acelerar determinadas búsquedas sobre documentos JSONB. pgAdmin 4 facilita la ejecución y exploración de la estructura.

MySQL

Se propone MySQL 8.0.16 o posterior por su uso extendido en aplicaciones web, su integración con herramientas como MySQL Workbench y el tipo JSON nativo. La identidad se expresa con AUTO_INCREMENT y las tablas usan InnoDB para mantener integridad referencial. Se debe confirmar la versión instalada, especialmente para validar restricciones CHECK.

Microsoft SQL Server

Se propone SQL Server 2016 o posterior por su integración con el ecosistema Microsoft y la herramienta SQL Server Management Studio (SSMS). En las versiones consideradas, el contenido JSON se almacena como NVARCHAR(MAX) y se valida con ISJSON. La identidad se declara con IDENTITY(1,1).

SGBD	Ventajas para el proyecto	Consideraciones
PostgreSQL	Código abierto; JSONB; índices GIN; restricciones robustas.	Requiere conocer operadores y funciones específicas de PostgreSQL.
MySQL	Tipo JSON; InnoDB; adopción amplia en aplicaciones web.	AUTO_INCREMENT y funciones JSON tienen sintaxis propia; verificar versión.
SQL Server	SSMS; integración con herramientas Microsoft; ISJSON.	JSON se representa como texto validado en las versiones consideradas; sintaxis propia.

Continuidad con el modelo de la Tarea 3

Se mantienen diez tablas: estudiante, perfil, publicacion, grupo, evento, interes, estudiante_grupo, estudiante_evento, estudiante_interes y seguidor. El diseño contempla trece claves foráneas, la relación uno a uno entre estudiante y perfil, relaciones muchos a muchos mediante tablas asociativas y una restricción que impide que un estudiante se siga a sí mismo.

Para alinear los archivos, se adoptaron los tamaños y reglas documentados en el PDF de la Tarea 3: estudiante.nombre y apellido VARCHAR(50), correo VARCHAR(100), programa VARCHAR(80), semestre SMALLINT; perfil.biografia VARCHAR(300), foto VARCHAR(150), contacto VARCHAR(100); publicacion.contenido VARCHAR(500); grupo.nombre_grupo VARCHAR(80) sin unicidad obligatoria; descripciones de grupo/evento VARCHAR(300); interes.nombre_interes VARCHAR(50) único y descripción VARCHAR(200).

Se corrigió la referencia de estudiante_evento.id_evento para apuntar a evento.id_evento.

Del modelo lógico al físico

Elemento	Decisión física
Identificadores	INT autogenerado: GENERATED BY DEFAULT AS IDENTITY (PostgreSQL), AUTO_INCREMENT (MySQL) e IDENTITY(1,1) (SQL Server).
Claves primarias compuestas	Se usan en las tablas asociativas y en seguidor para impedir relaciones duplicadas.
Integridad referencial	Las trece claves foráneas enlazan las tablas conforme al modelo. CASCADE se usa cuando la fila dependiente pierde sentido; RESTRICT/NO ACTION protege grupos y eventos creados por estudiantes.
Reglas de dominio	CHECK limita semestre a 1–12 y evita auto-seguimiento.
Fechas	TIMESTAMP en PostgreSQL; TIMESTAMP/DATETIME en MySQL; DATETIME2 en SQL Server, según la necesidad.
Datos semiestructurados	JSONB en PostgreSQL; JSON en MySQL; NVARCHAR(MAX) con ISJSON en SQL Server.

Índices y rendimiento

Las claves primarias y restricciones UNIQUE crean los índices necesarios según cada motor. PostgreSQL incluye un índice GIN sobre perfil.datos_extra para determinadas consultas JSONB. No se debe asumir que este índice es portable a MySQL o SQL Server: los índices sobre propiedades JSON deben definirse en función de las consultas y de las capacidades de la versión instalada. Antes de añadir índices a todas las claves foráneas, conviene revisar los planes de consulta y las necesidades reales de la aplicación.

Datos semiestructurados: JSON, Big Data e IoT

El atributo perfil.datos_extra permite registrar información opcional que puede variar entre estudiantes, como habilidades, deportes, idiomas o ubicación aproximada. Un ejemplo es un documento con una lista de habilidades y un objeto de ubicación que incluya latitud y longitud. Al ser atributos variables, no siempre conviene crear una columna relacional nueva por cada característica.

En un escenario relacionado con IoT, una aplicación universitaria podría recibir desde dispositivos móviles datos contextuales o coordenadas asociadas a actividades del campus. El documento JSON facilita conservar campos variables y consultarlos posteriormente. No obstante, JSON por sí solo no convierte la solución en una arquitectura Big Data ni sustituye un sistema de ingestión, controles de privacidad, validación de

coordenadas, políticas de retención o tipos geoespaciales especializados cuando estos sean necesarios.

Las implementaciones tienen diferencias: PostgreSQL usa JSONB, MySQL ofrece JSON nativo y SQL Server almacena el documento como texto Unicode validado con ISJSON en las versiones consideradas. El índice GIN es específico de PostgreSQL. Para comparar rendimiento, el equipo debe ejecutar consultas equivalentes sobre datos de prueba comparables.

Implementación paso a paso

1. **Revisión del modelo:** Se revisó el diseño de la base de datos de la tarea anterior para mantener la estructura de la Red Social Estudiantil Pascualina.
2. **Preparación de los scripts:** Se adaptaron los comandos necesarios para crear las tablas y relaciones en PostgreSQL, MySQL y SQL Server.
3. **Creación de las tablas:** Se ejecutaron los scripts en cada sistema, teniendo en cuenta las diferencias entre ellos y manteniendo la misma estructura de información.
4. **Validación de los datos:** Se realizaron pruebas para comprobar que las tablas, las relaciones y las restricciones funcionaran correctamente.
5. **Elaboración de los diccionarios:** Se organizaron los diccionarios de datos para documentar los campos, sus tipos y las relaciones de cada base de datos.
6. **Organización de la entrega:** Finalmente, se reunieron los scripts, los diccionarios y las evidencias para preparar el informe y organizar los archivos del proyecto en GitHub.

1. Ejecución del DDL en PostgreSQL

```sql
CREATE TABLE estudiante (
 id_estudiante INT GENERATED BY DEFAULT AS IDENTITY,
 nombre VARCHAR(50) NOT NULL,
 apellido VARCHAR(50) NOT NULL,
 correo VARCHAR(100) NOT NULL,
 programa VARCHAR(80) NOT NULL,
 semestre SMALLINT NOT NULL,
 CONSTRAINT pk_estudiante PRIMARY KEY (id_estudiante),
 CONSTRAINT uq_estudiante_correo UNIQUE (correo),
 CONSTRAINT chk_estudiante_semestre CHECK (semestre BETWEEN 1 AND 12)
);
CREATE TABLE interes (
 id_interes INT GENERATED BY DEFAULT AS IDENTITY,
 nombre_interes VARCHAR(50) NOT NULL,
 descripcion VARCHAR(200),
 CONSTRAINT pk_interes PRIMARY KEY (id_interes),
 CONSTRAINT uq_interes_nombre UNIQUE (nombre_interes)
);
CREATE TABLE perfil (
 id_perfil INT GENERATED BY DEFAULT AS IDENTITY,
 id_estudiante INT NOT NULL,
 biografia VARCHAR(300), foto VARCHAR(150), contacto VARCHAR(100),
 datos_extra JSONB NOT NULL DEFAULT '{}'::jsonb,
 CONSTRAINT pk_perfil PRIMARY KEY (id_perfil),
 CONSTRAINT uq_perfil_estudiante UNIQUE (id_estudiante),
 CONSTRAINT fk_perfil_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante) ON DELETE CASCADE ON UPDA
);
CREATE TABLE publicacion (
 id_publicacion INT GENERATED BY DEFAULT AS IDENTITY,
 id_estudiante INT NOT NULL, contenido VARCHAR(500) NOT NULL,
 fecha_hora TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT pk_publicacion PRIMARY KEY (id_publicacion),
 CONSTRAINT fk_publicacion_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante) ON DELETE CASCADE ON
);
CREATE TABLE grupo (
 id_grupo INT GENERATED BY DEFAULT AS IDENTITY,
 id_estudiante_creador INT NOT NULL, nombre_grupo VARCHAR(80) NOT NULL,
 descripcion VARCHAR(300), fecha_creacion DATE NOT NULL DEFAULT CURRENT_DATE,
 CONSTRAINT pk_grupo PRIMARY KEY (id_grupo),
 CONSTRAINT fk_grupo_creador FOREIGN KEY (id_estudiante_creador) REFERENCES estudiante(id_estudiante) ON DELETE RESTRICT ON
);
CREATE TABLE evento (
 id_evento INT GENERATED BY DEFAULT AS IDENTITY,
 id_estudiante_organizador INT NOT NULL, id_grupo INT NULL,
 nombre_evento VARCHAR(100) NOT NULL, descripcion VARCHAR(300),
 fecha_hora TIMESTAMP NOT NULL, ubicacion VARCHAR(150) NOT NULL,
 CONSTRAINT pk_evento PRIMARY KEY (id_evento),
 CONSTRAINT fk_evento_organizador FOREIGN KEY (id_estudiante_organizador) REFERENCES estudiante(id_estudiante) ON DELETE RE
 CONSTRAINT fk_evento_grupo FOREIGN KEY (id_grupo) REFERENCES grupo(id_grupo) ON DELETE CASCADE ON UPDATE CASCADE
);
CREATE TABLE estudiante_grupo (
 id_estudiante INT NOT NULL, id_grupo INT NOT NULL,
 CONSTRAINT pk_estudiante_grupo PRIMARY KEY (id_estudiante,id_grupo),
 CONSTRAINT fk_estudiante_grupo_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante) ON DELETE CASCA
 CONSTRAINT fk_estudiante_grupo_grupo FOREIGN KEY (id_grupo) REFERENCES grupo(id_grupo) ON DELETE CASCADE ON UPDATE CASCADE
);
CREATE TABLE estudiante_evento (
 id_estudiante INT NOT NULL, id_evento INT NOT NULL,
 CONSTRAINT pk_estudiante_evento PRIMARY KEY (id_estudiante,id_evento),
 CONSTRAINT fk_estudiante_evento_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante) ON DELETE CASC
 CONSTRAINT fk_estudiante_evento_evento FOREIGN KEY (id_evento) REFERENCES evento(id_evento) ON DELETE CASCADE ON UPDATE CA
);
CREATE TABLE estudiante_interes (
 id_estudiante INT NOT NULL, id_interes INT NOT NULL,
 CONSTRAINT pk_estudiante_interes PRIMARY KEY (id_estudiante,id_interes),
 CONSTRAINT fk_estudiante_interes_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante) ON DELETE CAS
 CONSTRAINT fk_estudiante_interes_interes FOREIGN KEY (id_interes) REFERENCES interes(id_interes) ON DELETE CASCADE ON UPDA
);
CREATE TABLE seguidor (
 id_seguidor INT NOT NULL, id_seguido INT NOT NULL,
 CONSTRAINT pk_seguidor PRIMARY KEY (id_seguidor,id_seguido),
 CONSTRAINT chk_seguidor_no_propio CHECK (id_seguidor <> id_seguido),
 CONSTRAINT fk_seguidor_seguidor FOREIGN KEY (id_seguidor) REFERENCES estudiante(id_estudiante) ON DELETE CASCADE ON UPDATE
```

```
74    CONSTRAINT fk_seguidor_seguido FOREIGN KEY (id_seguido) REFERENCES estudiante(id_estudiante) ON DELETE CASCADE ON UPDATE C
75  );
```

```
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE INDEX
```

```
1  SELECT table_name
2  FROM information_schema.tables
3  WHERE table_schema = 'public'
4    AND table_type = 'BASE TABLE'
5  ORDER BY table_name;
```

nombre_de_tabla
estudiante
evento_estudiante
estudiante_grupo
estudiante_interes
evento
grupo
intereses
perfil
publicación
seguidor

```
SELECT 10
```

RESULTADOS DE CONSULTAS Y DATOS DE PRUEBA – RESTRICCIONES DE DATOS INVALIDOS

```
1  BEGIN;
2  INSERT INTO estudiante(nombre,apellido,correo,programa,semestre) VALUES
3  ('Ana','Prueba','ana.prueba@ejemplo.com','Ingeniería de Sistemas',3),
4  ('Luis','Prueba','luis.prueba@ejemplo.com','Tecnología en Sistemas',5);
5  INSERT INTO perfil(id_estudiante,biografia,contacto,datos_extra) VALUES
6  (1,'Perfil de prueba','ana@ejemplo.com','{"habilidades":["SQL","Python"],"deportes":["fút
7  SELECT id_estudiante,nombre,correo FROM estudiante ORDER BY id_estudiante;
8  SELECT id_estudiante,datos_extra->'habilidades' AS habilidades,datos_extra->'ubicacion'-
9  SELECT count(*) AS tablas FROM information_schema.tables WHERE table_schema='public' AND
10 SELECT count(*) AS claves_foraneas FROM information_schema.table_constraints WHERE constr
11 -- Las siguientes pruebas negativas deben ejecutarse individualmente y se espera que fal
12 -- INSERT INTO estudiante(nombre,apellido,correo,programa,semestre) VALUES ('X','Y','x@e
13 -- INSERT INTO seguidor(id_seguidor,id_seguido) VALUES (1,1); -- CHECK no seguirse a sí
14 -- INSERT INTO estudiante(nombre,apellido,correo,programa,semestre) VALUES ('Duplicado',
15 ROLLBACK;
```

```
BEGIN
INSERT 0 2
INSERT 0 1
```

id_estudiante	nombre	correo
1	Ana	ana.prueba@ejemplo.com
2	Luis	luis.prueba@ejemplo.com

```
SELECT 2
```

id_estudiante	habilidades	latitud
1	["SQL", "Python"]	6.24

```
SELECT 1
```

tablas
10

```
SELECT 1
```

claves_foráneas
13

```
SELECT 1
ROLLBACK
```

```
1  -- PRUEBA 1: INSERTAR UN ESTUDIANTE VÁLIDO
2  INSERT INTO estudiante
3      (nombre, apellido, correo, programa, semestre)
4  VALUES
5      ('Ana', 'Prueba', 'ana.prueba@example.com',
6       'Ingeniería de Sistemas', 5);
7
```

```
INSERT 0 1
```

```
1  -- PRUEBA 2: INTENTAR REPETIR UN CORREO (UNIQUE)
2  INSERT INTO estudiante
3      (nombre, apellido, correo, programa, semestre)
4  VALUES
5      ('Luis', 'Prueba', 'ana.prueba@example.com',
6       'Ingeniería de Sistemas', 4);
7
8
```

```
ERROR:  duplicate key value violates unique constraint "uq_estudiante_correo"
DETAIL:  Key (correo)=(ana.prueba@example.com) already exists.
```

```
1  -- PRUEBA 3: SEMESTRE FUERA DEL RANGO (CHECK)
2  INSERT INTO estudiante
3      (nombre, apellido, correo, programa, semestre)
4  VALUES
5      ('Carlos', 'Prueba', 'carlos.prueba@example.com',
6       'Ingeniería de Sistemas', 15);
```

```
ERROR:  new row for relation "estudiante" violates check constraint "chk_estudiante_semestre"
DETAIL:  Failing row contains (5, Carlos, Prueba, carlos.prueba@example.com, Ingeniería de Sistemas, 15).
```

Importante: el error no significa que el script esté mal. En esta prueba, el rechazo es el resultado esperado.

2. Ejecución del DDL en SQL server



```
CREATE TABLE dbo.estudiante (
 id_estudiante INT IDENTITY(1,1) NOT NULL,
 nombre VARCHAR(50) NOT NULL, apellido VARCHAR(50) NOT NULL,
 correo VARCHAR(100) NOT NULL, programa VARCHAR(80) NOT NULL,
 semestre SMALLINT NOT NULL,
 CONSTRAINT pk_estudiante PRIMARY KEY (id_estudiante),
 CONSTRAINT uq_estudiante_correo UNIQUE (correo),
 CONSTRAINT chk_estudiante_semestre CHECK (semestre BETWEEN 1 AND 12)
);
CREATE TABLE dbo.interes (
 id_interes INT IDENTITY(1,1) NOT NULL, nombre_interes VARCHAR(50) NOT NULL,
 descripcion VARCHAR(200), CONSTRAINT pk_interes PRIMARY KEY (id_interes),
 CONSTRAINT uq_interes_nombre UNIQUE (nombre_interes)
);
CREATE TABLE dbo.perfil (
 id_perfil INT IDENTITY(1,1) NOT NULL, id_estudiante INT NOT NULL,
 biografia VARCHAR(300), foto VARCHAR(150), contacto VARCHAR(100),
 datos_extra NVARCHAR(MAX) NOT NULL CONSTRAINT df_perfil_datos_extra DEFAULT N'{}',
 CONSTRAINT pk_perfil PRIMARY KEY (id_perfil), CONSTRAINT uq_perfil_estudiante UNIQUE (id_es
 CONSTRAINT ck_perfil_datos_extra_json CHECK (ISJSON(datos_extra)=1),
 CONSTRAINT fk_perfil_estudiante FOREIGN KEY (id_estudiante) REFERENCES dbo.estudiante(id_es
);
CREATE TABLE dbo.publicacion (
 id_publicacion INT IDENTITY(1,1) NOT NULL, id_estudiante INT NOT NULL,
 contenido VARCHAR(500) NOT NULL, fecha_hora DATETIME2 NOT NULL CONSTRAINT df_publicacion_fe
 CONSTRAINT pk_publicacion PRIMARY KEY (id_publicacion),
 CONSTRAINT fk_publicacion_estudiante FOREIGN KEY (id_estudiante) REFERENCES dbo.estudiante(
);
CREATE TABLE dbo.grupo (
 id_grupo INT IDENTITY(1,1) NOT NULL, id_estudiante_creador INT NOT NULL,
 nombre_grupo VARCHAR(80) NOT NULL, descripcion VARCHAR(300), fecha_creacion DATE NOT NULL C
 CONSTRAINT pk_grupo PRIMARY KEY (id_grupo),
 CONSTRAINT fk_grupo_creador FOREIGN KEY (id_estudiante_creador) REFERENCES dbo.estudiante(i
);
CREATE TABLE dbo.evento (
 id_evento INT IDENTITY(1,1) NOT NULL, id_estudiante_organizador INT NOT NULL, id_grupo INT
 nombre_evento VARCHAR(100) NOT NULL, descripcion VARCHAR(300), fecha_hora DATETIME2 NOT NUL
 CONSTRAINT pk_evento PRIMARY KEY (id_evento),
 CONSTRAINT fk_evento_organizador FOREIGN KEY (id_estudiante_organizador) REFERENCES dbo.est
 CONSTRAINT fk_evento_grupo FOREIGN KEY (id_grupo) REFERENCES dbo.grupo(id_grupo) ON DELETE
);
CREATE TABLE dbo.estudiante_grupo (
 id_estudiante INT NOT NULL, id_grupo INT NOT NULL, CONSTRAINT pk_estudiante_grupo PRIMARY K
 CONSTRAINT fk_estudiante_grupo_estudiante FOREIGN KEY (id_estudiante) REFERENCES dbo.estudi
 CONSTRAINT fk_estudiante_grupo_grupo FOREIGN KEY (id_grupo) REFERENCES dbo.grupo(id_grupo)
);
CREATE TABLE dbo.estudiante_evento (
 id_estudiante INT NOT NULL, id_evento INT NOT NULL, CONSTRAINT pk_estudiante_evento PRIMARY
 CONSTRAINT fk_estudiante_evento_estudiante FOREIGN KEY (id_estudiante) REFERENCES dbo.estud
 CONSTRAINT fk_estudiante_evento_evento FOREIGN KEY (id_evento) REFERENCES dbo.evento(id_eve
);
CREATE TABLE dbo.estudiante_interes (
 id_estudiante INT NOT NULL, id_interes INT NOT NULL, CONSTRAINT pk_estudiante_interes PRIMA
 CONSTRAINT fk_estudiante_interes_estudiante FOREIGN KEY (id_estudiante) REFERENCES dbo.estu
 CONSTRAINT fk_estudiante_interes_interes FOREIGN KEY (id_interes) REFERENCES dbo.interes(id_
);
CREATE TABLE dbo.seguidor (
 id_seguidor INT NOT NULL, id_seguido INT NOT NULL, CONSTRAINT pk_seguidor PRIMARY KEY (id_s
 CONSTRAINT chk_seguidor_no_propio CHECK (id_seguidor <> id_seguido),
 CONSTRAINT fk_seguidor_seguidor FOREIGN KEY (id_seguidor) REFERENCES dbo.estudiante(id_estu
 CONSTRAINT fk_seguidor_seguido FOREIGN KEY (id_seguido) REFERENCES dbo.estudiante(id_estudi
);
```

```sql
SELECT TABLE_NAME
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_TYPE = 'BASE TABLE'
ORDER BY TABLE_NAME;
```

TABLE_NAME
estudiante
estudiante_evento
estudiante_grupo
estudiante_interes
evento
grupo
interes
perfil
publicacion
seguidor

RESULTADOS DE CONSULTAS Y DATOS DE PRUEBA – RESTRICCIONES DE DATOS INVALIDOS

```sql
BEGIN TRANSACTION;
INSERT INTO dbo.estudiante(nombre, apellido, correo, programa, semestre)
VALUES ('Ana','Prueba','ana.prueba@example.test','Ingeniería de Sistemas',3),
       ('Luis','Prueba','luis.prueba@example.test','Tecnología en Sistemas',5);
INSERT INTO dbo.perfil(id_estudiante, biografia, contacto, datos_extra)
SELECT id_estudiante, 'Perfil de prueba', 'ana@example.test',
       N'{"habilidades":["SQL","Python"],"ubicacion":{"latitud":6.24,"longitud":-75.58}}'
FROM dbo.estudiante WHERE correo='ana.prueba@example.test';
SELECT id_estudiante, nombre, correo FROM dbo.estudiante WHERE correo LIKE '%.prueba@exar
SELECT p.id_estudiante, JSON_QUERY(p.datos_extra,'$.habilidades') AS habilidades,
       JSON_VALUE(p.datos_extra,'$.ubicacion.latitud') AS latitud
FROM dbo.perfil p JOIN dbo.estudiante e ON e.id_estudiante=p.id_estudiante
WHERE e.correo='ana.prueba@example.test';
SELECT COUNT(*) AS tablas FROM sys.tables;
SELECT COUNT(*) AS claves_foraneas FROM sys.foreign_keys;
ROLLBACK TRANSACTION;
```

id_estudiante	nombre	correo
1	Ana	ana.prueba@example.test
2	Luis	luis.prueba@example.test

id_estudiante	habilidades	latitud
1	["SQL","Python"]	6.24

tablas
10

claves_foraneas
13

```sql
INSERT INTO dbo.estudiante(nombre,apellido,correo,programa,semestre) VALUES ('X','Y','se
```

echarts

nothing to render: expected a table of one row and one column

```
error
Msg 547 Level 16 State 0 Line 1
The INSERT statement conflicted with the CHECK constraint "chk_estudiante_semestre". The co
Msg 3621 Level 0 State 0 Line 1
The statement has been terminated.
```

```sql
INSERT INTO dbo.estudiante(nombre,apellido,correo,programa,semestre) VALUES ('X','Y','an
```

echarts

nothing to render: expected a table of one row and one column

```
status
1 rows affected
```

```sql
INSERT INTO dbo.seguidor(id_seguidor,id_seguido) SELECT id_estudiante,id_estudiante FROM
```

echarts

nothing to render: expected a table of one row and one column

```
error
Msg 547 Level 16 State 0 Line 1
The INSERT statement conflicted with the CHECK constraint "chk_seguidor_no_propio". The con
Msg 3621 Level 0 State 0 Line 1
The statement has been terminated.
```

3. Ejecución del DDL en MySQL

```sql
CREATE TABLE estudiante (
 id_estudiante INT NOT NULL AUTO_INCREMENT,
 nombre VARCHAR(50) NOT NULL, apellido VARCHAR(50) NOT NULL,
 correo VARCHAR(100) NOT NULL, programa VARCHAR(80) NOT NULL,
 semestre SMALLINT NOT NULL,
 CONSTRAINT pk_estudiante PRIMARY KEY (id_estudiante),
 CONSTRAINT uq_estudiante_correo UNIQUE (correo),
 CONSTRAINT chk_estudiante_semestre CHECK (semestre BETWEEN 1 AND 12)
) ENGINE=InnoDB;
CREATE TABLE interes (
 id_interes INT NOT NULL AUTO_INCREMENT, nombre_interes VARCHAR(50) NOT NULL,
 descripcion VARCHAR(200), CONSTRAINT pk_interes PRIMARY KEY (id_interes),
 CONSTRAINT uq_interes_nombre UNIQUE (nombre_interes)
) ENGINE=InnoDB;
CREATE TABLE perfil (
 id_perfil INT NOT NULL AUTO_INCREMENT, id_estudiante INT NOT NULL,
 biografia VARCHAR(300), foto VARCHAR(150), contacto VARCHAR(100),
 datos_extra JSON NOT NULL DEFAULT (JSON_OBJECT()),
 CONSTRAINT pk_perfil PRIMARY KEY (id_perfil), CONSTRAINT uq_perfil_estudiante UNIQUE (id
 CONSTRAINT fk_perfil_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_est
) ENGINE=InnoDB;
CREATE TABLE publicacion (
 id_publicacion INT NOT NULL AUTO_INCREMENT, id_estudiante INT NOT NULL,
 contenido VARCHAR(500) NOT NULL, fecha_hora TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
 CONSTRAINT pk_publicacion PRIMARY KEY (id_publicacion),
 CONSTRAINT fk_publicacion_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudiante(i
) ENGINE=InnoDB;
CREATE TABLE grupo (
 id_grupo INT NOT NULL AUTO_INCREMENT, id_estudiante_creador INT NOT NULL,
 nombre_grupo VARCHAR(80) NOT NULL, descripcion VARCHAR(300), fecha_creacion DATE NOT NUL
 CONSTRAINT pk_grupo PRIMARY KEY (id_grupo),
 CONSTRAINT fk_grupo_creador FOREIGN KEY (id_estudiante_creador) REFERENCES estudiante(id
) ENGINE=InnoDB;
CREATE TABLE evento (
 id_evento INT NOT NULL AUTO_INCREMENT, id_estudiante_organizador INT NOT NULL, id_grupo
 nombre_evento VARCHAR(100) NOT NULL, descripcion VARCHAR(300), fecha_hora DATETIME NOT N
 CONSTRAINT pk_evento PRIMARY KEY (id_evento),
 CONSTRAINT fk_evento_organizador FOREIGN KEY (id_estudiante_organizador) REFERENCES estu
 CONSTRAINT fk_evento_grupo FOREIGN KEY (id_grupo) REFERENCES grupo(id_grupo) ON DELETE (
) ENGINE=InnoDB;
CREATE TABLE estudiante_grupo (
 id_estudiante INT NOT NULL, id_grupo INT NOT NULL, CONSTRAINT pk_estudiante_grupo PRIMAR
 CONSTRAINT fk_estudiante_grupo_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudia
 CONSTRAINT fk_estudiante_grupo_grupo FOREIGN KEY (id_grupo) REFERENCES grupo(id_grupo) (
) ENGINE=InnoDB;
```

```sql
CREATE TABLE evento (
 id_evento INT NOT NULL AUTO_INCREMENT, id_estudiante_organizador INT NOT NULL, id_grupo
 nombre_evento VARCHAR(100) NOT NULL, descripcion VARCHAR(300), fecha_hora DATETIME NOT N
 CONSTRAINT pk_evento PRIMARY KEY (id_evento),
 CONSTRAINT fk_evento_organizador FOREIGN KEY (id_estudiante_organizador) REFERENCES estu
 CONSTRAINT fk_evento_grupo FOREIGN KEY (id_grupo) REFERENCES grupo(id_grupo) ON DELETE C
) ENGINE=InnoDB;
CREATE TABLE estudiante_grupo (
 id_estudiante INT NOT NULL, id_grupo INT NOT NULL, CONSTRAINT pk_estudiante_grupo PRIMAR
 CONSTRAINT fk_estudiante_grupo_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudia
 CONSTRAINT fk_estudiante_grupo_grupo FOREIGN KEY (id_grupo) REFERENCES grupo(id_grupo) C
) ENGINE=InnoDB;
CREATE TABLE estudiante_evento (
 id_estudiante INT NOT NULL, id_evento INT NOT NULL, CONSTRAINT pk_estudiante_evento PRIM
 CONSTRAINT fk_estudiante_evento_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudi
 CONSTRAINT fk_estudiante_evento_evento FOREIGN KEY (id_evento) REFERENCES evento(id_even
) ENGINE=InnoDB;
CREATE TABLE estudiante_interes (
 id_estudiante INT NOT NULL, id_interes INT NOT NULL, CONSTRAINT pk_estudiante_interes PR
 CONSTRAINT fk_estudiante_interes_estudiante FOREIGN KEY (id_estudiante) REFERENCES estud
 CONSTRAINT fk_estudiante_interes_interes FOREIGN KEY (id_interes) REFERENCES interes(id_
) ENGINE=InnoDB;
CREATE TABLE seguidor (
    id_seguidor INT NOT NULL, id_seguido INT NOT NULL, CONSTRAINT pk_seguidor PRIMARY KEY
  CONSTRAINT fk_seguidor_seguidor FOREIGN KEY (id_seguidor) REFERENCES estudiante(id_estu
  CONSTRAINT fk_seguidor_seguido FOREIGN KEY (id_seguido) REFERENCES estudiante(id_estudi
  CONSTRAINT chk_seguidor_no_propio CHECK (id_seguidor <> id_seguido)
) ENGINE=InnoDB;
```

```sql
SHOW TABLES;
```

Tables_in_fiddle
estudiante
estudiante_evento
estudiante_grupo
estudiante_interes
evento
grupo
interes
perfil
publicacion
seguidor

RESULTADOS DE CONSULTAS Y DATOS DE PRUEBA – RESTRICCIONES DE DATOS INVALIDOS

```sql
-- 1. INSERTAR ESTUDIANTES
INSERT INTO estudiante
    (nombre, apellido, correo, programa, semestre)
VALUES
    ('Ana', 'Prueba', 'ana.consulta@example.com',
     'Ingenieria de Sistemas', 5),
    ('Luis', 'Prueba', 'luis.consulta@example.com',
     'Ingenieria de Sistemas', 4);

-- 2. INSERTAR PERFILES
INSERT INTO perfil
    (id_estudiante, biografia, foto, contacto, datos_extra)
VALUES
    (1, 'Estudiante interesada en tecnologia',
     'ana.jpg', 'ana.contacto@example.com',
     JSON_OBJECT('habilidades', JSON_ARRAY('SQL', 'Python'))),
    (2, 'Estudiante interesado en desarrollo',
```

<>fiddle MySQL 8.0 no sample DB run markdown clear donate feedback about

```sql

-- 3. INSERTAR PUBLICACIONES
INSERT INTO publicacion
    (id_estudiante, contenido)
VALUES
    (1, 'Bienvenidos a la red social estudiantil'),
    (2, 'Busco compañeros para estudiar bases de datos');

-- 4. INSERTAR GRUPOS
INSERT INTO grupo
    (id_estudiante_creador, nombre_grupo, descripcion)
VALUES
```

```sql
    (1, 'Grupo de Bases de Datos',
     'Espacio para aprender SQL'),
    (2, 'Grupo de Programacion',
     'Comunidad de estudiantes programadores');

-- 5. INSERTAR EVENTOS
INSERT INTO evento
    (id_estudiante_organizador, id_grupo, nombre_evento,
     descripcion, fecha_hora, ubicacion)
VALUES
    (1, 1, 'Taller de SQL',
     'Practica de consultas SQL', '2026-11-15 10:00:00',
     'Laboratorio 1'),
    (2, 2, 'Encuentro de programacion',
     'Actividad de aprendizaje colaborativo',
     '2026-11-20 14:00:00', 'Aula 2');

-- 6. INSERTAR INTERESES
INSERT INTO interes (nombre_interes, descripcion)
VALUES
    ('Bases de datos', 'Modelado y consultas SQL'),
    ('Programacion', 'Desarrollo de software');

-- 7. RELACIONAR ESTUDIANTES CON GRUPOS
INSERT INTO estudiante_grupo (id_estudiante, id_grupo)
VALUES (1, 1), (2, 2);

-- 8. RELACIONAR ESTUDIANTES CON EVENTOS
INSERT INTO estudiante_evento (id_estudiante, id_evento)
VALUES (1, 1), (2, 2);

-- 9. RELACIONAR ESTUDIANTES CON INTERESES
INSERT INTO estudiante_interes (id_estudiante, id_interes)
VALUES (1, 1), (2, 2);

-- 10. INSERTAR SEGUIDORES
INSERT INTO seguidor (id_seguidor, id_seguido)
VALUES (1, 2), (2, 1);
```

```sql

-- Evidencia 1: estudiantes
SELECT * FROM estudiante;

-- Evidencia 2: perfiles
SELECT * FROM perfil;

-- Evidencia 3: publicaciones
SELECT * FROM publicacion;

-- Evidencia 4: grupos
SELECT * FROM grupo;

-- Evidencia 5: eventos
SELECT * FROM evento;

-- Evidencia 6: intereses
SELECT * FROM interes;

-- Evidencia 7: estudiantes y grupos
SELECT * FROM estudiante_grupo;

-- Evidencia 8: estudiantes y eventos
SELECT * FROM estudiante_evento;

-- Evidencia 9: estudiantes e intereses
SELECT * FROM estudiante_interes;

-- Evidencia 10: seguidores
```

```
Records: 2  Duplicates: 0  Warnings: 0
Records: 2  Duplicates: 0  Warnings: 0
Records: 2  Duplicates: 0  Warnings: 0
Records: 2  Duplicates: 0  Warnings: 0
Records: 2  Duplicates: 0  Warnings: 0
Records: 2  Duplicates: 0  Warnings: 0
Records: 2  Duplicates: 0  Warnings: 0
Records: 2  Duplicates: 0  Warnings: 0
```

```
Records: 2  Duplicates: 0  Warnings: 0
Records: 2  Duplicates: 0  Warnings: 0
```

id_estudiante	nombre	apellido	correo	programa	semestre
1	Ana	Prueba	ana.consulta@example.com	Ingenieria de Sistemas	5
2	Luis	Prueba	luis.consulta@example.com	Ingenieria de Sistemas	4

id_perfil	id_estudiante	biografia	foto	contacto	datos_extra
1	1	Estudiante interesada en tecnologia	ana.jpg	ana.contacto@example.com	{"habilidades": ["SQL", "Python"]}
2	2	Estudiante interesado en desarrollo	luis.jpg	luis.contacto@example.com	{"habilidades": ["Java", "MySQL"]}

id_publicacion	id_estudiante	contenido	fecha_hora
1	1	Bienvenidos a la red social estudiantil	2026-10-26 06:17:35
2	2	Busco compañeros para estudiar bases de datos	2026-10-26 06:17:35

id_grupo	id_estudiante_creador	nombre_grupo	descripcion	fecha_creacion
1	1	Grupo de Bases de Datos	Espacio para aprender SQL	2026-10-26
2	2	Grupo de Programacion	Comunidad de estudiantes programadores	2026-10-26

id_evento	id_estudiante_organizador	id_grupo	nombre_evento	descripcion	fecha_hora	ubicacion
1	1	1	Taller de SQL	Practica de consultas SQL	2026-11-15 10:00:00	Laboratorio 1
2	2	2	Encuentro de programacion	Actividad de aprendizaje colaborativo	2026-11-20 14:00:00	Aula 2

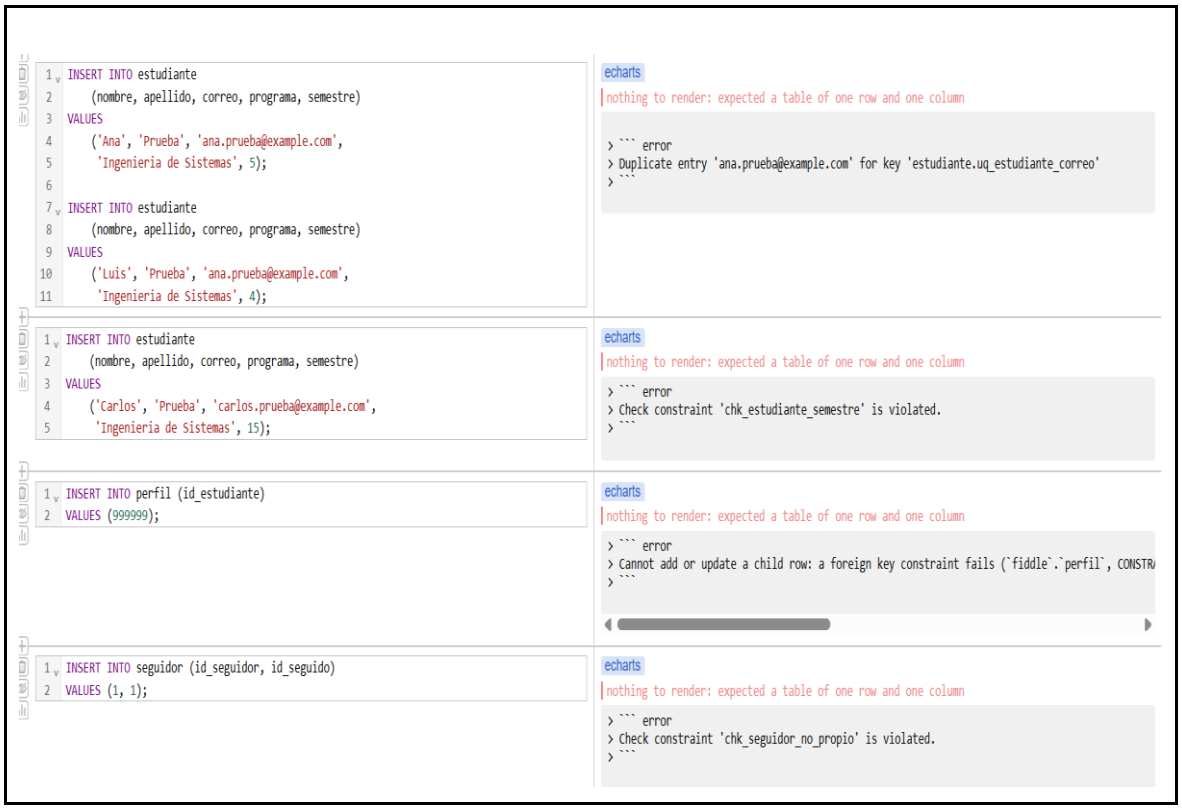
id_interes	nombre_interes	descripcion
1	Bases de datos	Modelado y consultas SQL
2	Programacion	Desarrollo de software

id_estudiante	id_grupo
1	1
2	2

id_estudiante	id_evento
1	1
2	2

id_estudiante	id_interes
1	1
2	2

id_seguidor	id_seguido
2	1
1	2



Dificultades, ventajas y desventajas

1. PostgreSQL

Dificultades

Durante la implementación del modelo de la Red Social Estudiantil Pascualina se requirió verificar la definición de las claves primarias, claves foráneas y restricciones de integridad. También fue necesario comprobar el comportamiento de las restricciones UNIQUE y CHECK mediante datos de prueba. Una dificultad adicional fue trabajar con el tipo de dato JSONB para almacenar información semiestructurada en los perfiles de los estudiantes.

Ventajas

- Permite definir restricciones de integridad para mantener la consistencia de los datos.
- Ofrece el tipo de dato JSONB, útil para almacenar información semiestructurada y consultar su contenido.
- Soporta relaciones entre tablas mediante claves foráneas y permite establecer acciones referenciales.
- Es un sistema gestor de código abierto que ofrece funcionalidades avanzadas para el manejo de datos relacionales.

Desventajas

- Su sintaxis y algunos tipos de datos son diferentes de los utilizados en otros motores, por lo que los scripts requieren adaptaciones para migrarlos.
- El uso de características específicas, como JSONB, puede dificultar la portabilidad directa del modelo a otros sistemas gestores.
- Es necesario conocer sus particularidades para aprovechar adecuadamente sus funciones y herramientas de administración.

2. MySQL

Dificultades

Durante la implementación en MySQL fue necesario adaptar la sintaxis del script DDL, especialmente las instrucciones de autoincremento, la definición de tablas y el almacenamiento de información JSON. También se presentó un conflicto al combinar la restricción CHECK que impide que un estudiante se siga a sí mismo con las claves foráneas y sus acciones referenciales. Para resolverlo, se ajustó la definición de la tabla seguidor, conservando las reglas de integridad del modelo.

Ventajas

- Utiliza una sintaxis ampliamente empleada en aplicaciones web.
- Permite definir claves primarias, claves foráneas y restricciones para proteger la integridad de los datos.
- Incluye el tipo de dato JSON para almacenar información semiestructurada.
- Es de uso extendido y dispone de herramientas para administrar bases de datos relacionales.

Desventajas

- La compatibilidad de algunas características depende de la versión del motor.
- Las diferencias frente a PostgreSQL y SQL Server requieren adaptar los scripts DDL.
- Algunas combinaciones de restricciones y acciones referenciales pueden requerir ajustes específicos para funcionar correctamente.

3. Microsoft SQL Server

Dificultades

Durante la adaptación del modelo a SQL Server se identificaron diferencias de sintaxis respecto a MySQL y PostgreSQL. Por ejemplo, SQL Server no utiliza AUTO_INCREMENT ni la cláusula ENGINE=InnoDB; para los identificadores autoincrementales utiliza IDENTITY(1,1). Por esta razón, fue necesario adaptar el script para que las tablas, los tipos de datos y las restricciones fueran compatibles con el motor seleccionado.

Ventajas

- Permite definir claves primarias, claves foráneas y restricciones de integridad.
- Proporciona herramientas de administración y consulta mediante SQL Server Management Studio.
- Permite implementar soluciones relacionales y trabajar con información semiestructurada mediante las funcionalidades JSON disponibles en el motor y la versión utilizados.
- Ofrece herramientas para administrar, consultar y mantener bases de datos.

Desventajas

- La sintaxis del DDL presenta diferencias frente a PostgreSQL y MySQL, por lo que no es posible trasladar todos los scripts directamente.
- Algunas funcionalidades dependen de la versión y edición de SQL Server.
- Su entorno de administración y configuración puede requerir familiarización por parte de quienes trabajan principalmente con otros motores.

4. Análisis comparativo

La implementación del mismo modelo en los tres sistemas gestores permitió identificar que las tablas, las relaciones y las reglas de integridad pueden conservar su propósito aunque cambie la sintaxis utilizada. Sin embargo, las diferencias en autoincremento, tipos de datos, almacenamiento JSON y acciones referenciales hacen necesario adaptar cada script DDL.

PostgreSQL proporciona el tipo JSONB; MySQL dispone del tipo JSON; y SQL Server permite trabajar con información JSON mediante las funciones y capacidades disponibles en la versión utilizada. Estas diferencias deben tenerse en cuenta al diseñar soluciones que necesiten almacenar datos semiestructurados y mantener la portabilidad entre motores.

En conclusión, la selección de un sistema gestor debe considerar los requisitos del proyecto, las funcionalidades necesarias, la experiencia del equipo y el entorno en el que se implementará la solución. Para la Red Social Estudiantil Pascualina, la comparación permite comprender la importancia de documentar las adaptaciones y comprobar las restricciones en cada motor.

Conclusiones

Conclusión grupal

El proceso evidencia que un modelo lógico común puede implementarse en varios SGBD, pero cada motor exige adaptar la sintaxis, los tipos de datos y algunas reglas de integridad. Mantener una fuente común para las entidades y documentar las diferencias físicas ayuda a reducir inconsistencias entre scripts y diccionarios. El uso de JSON permite representar atributos variables del perfil, aunque debe acompañarse de validación, criterios de privacidad e índices diseñados para consultas reales.

Conclusiones individuales

Mariana Díaz Lastre: La realización de esta actividad me permitió comprender la importancia de adaptar un modelo de datos a las características particulares de diferentes sistemas gestores de bases de datos. Al comparar MySQL y SQL Server con PostgreSQL, pude reconocer que, aunque los tres permiten implementar estructuras relacionales, presentan diferencias en la sintaxis, los tipos de datos y el manejo de determinadas restricciones. También comprendí la importancia de mantener la coherencia entre el modelo físico, los scripts DDL y los diccionarios de datos. Como aprendizaje, destaco la necesidad de realizar pruebas de funcionamiento y documentar las ventajas, limitaciones y posibles mejoras de cada implementación para obtener una solución confiable y adecuada a las necesidades de la red social estudiantil.

José David Celis Cepeda: El desarrollo de esta actividad me permitió fortalecer mis conocimientos sobre el diseño e implementación de bases de datos relacionales mediante PostgreSQL. Comprendí la importancia de definir correctamente las tablas, los tipos de datos, las claves primarias y foráneas y las restricciones de integridad para garantizar la coherencia de la información. Asimismo, reconocí la utilidad del tipo de dato JSONB para almacenar información semiestructurada en los perfiles de los estudiantes, facilitando la incorporación de atributos adicionales. Esta experiencia me permitió comprender que el diseño de una base de datos no consiste únicamente en crear tablas, sino también en documentar las decisiones técnicas y comprobar que la estructura responda a las necesidades del sistema.

Propuestas de mejora

- Se propone hacer que las consultas sean más rápidas, revisar que los datos estén bien escritos y cuidar mejor la información de los estudiantes. También es importante mantener los archivos del proyecto al día y guardar copias de seguridad para evitar perder información.
- Estas mejoras ayudarían a que la Red Social Estudiantil Pascualina funcione mejor y pueda crecer en el futuro.